

Adding Supabase Auth to a Next.js + FastAPI App

A reusable setup guide — written from a working implementation (Analytika, 2026-07-04). Stack: **Next.js 16 (App Router) + Tailwind v4 + shadcn/ui** frontend · **FastAPI** backend · **Supabase** (Auth + Postgres). Auth method: **email + password**.

This document is meant to be handed to a future Claude Code session (or your future self) to reproduce the same setup in another project. Follow it top to bottom.

⚠️ READ THIS FIRST — the JWT signing change

Supabase moved from a single **Legacy JWT Secret** (HS256, one shared secret) to **JWT Signing Keys** (asymmetric — typically **ECC P-256 / ES256**). On a migrated project:

- **User access tokens are now signed with the current asymmetric key (ES256)**. You verify them with the project's **public keys** from the **JWKS endpoint** — *not* a shared secret.
- The "**Legacy JWT Secret (still used)**" shown in the dashboard is retained **only** to verify old/legacy tokens and the legacy `anon` / `service_role` API keys. **It does NOT sign your users' login tokens anymore.**

Consequences for the backend:

1. **Do NOT verify with `SUPABASE_JWT_SECRET` (HS256)**. That's the old way; it will reject ES256 tokens with a `401`.
2. **Verify via JWKS** at `https://<project-ref>.supabase.co/auth/v1/.well-known/jwks.json`.
3. You do **not** need to set a JWT secret env var at all — verification only needs `SUPABASE_URL`.

If you find yourself pasting a "JWT Secret" into the backend `.env`, stop — that's the deprecated path.

1. Architecture

- **Frontend** authenticates directly with **Supabase Auth** (`@supabase/ssr`). The browser holds the session in cookies.
- On every call to the backend, the frontend attaches the Supabase **access token** as `Authorization: Bearer <jwt>`.
- **Backend (FastAPI)** verifies that token against Supabase's **public JWKS keys**, extracts the user id (`sub`), and scopes all data to that user.
- **Postgres**: a `user_id` column links rows to `auth.users`, plus **Row-Level Security (RLS)** as defense-in-depth. The backend uses the **service_role** key, which bypasses RLS and is the primary gatekeeper.

Browser —(Supabase Auth: email+password)—> Supabase

|

| Authorization: Bearer <ES256 access token>

▼

FastAPI —(verify token via JWKS public keys)—> extract user.id —> scope data

2. The keys — what goes where

Key	Location	Exposure	Purpose
anon (public)	<code>frontend/.env.local</code> → <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Ships to the browser	Client-side auth; constrained by RLS
service_role (secret)	<code>backend/.env</code> → <code>SUPABASE_KEY</code>	Server only — never in frontend	Backend DB access; bypasses RLS
Project URL	both	Public	API + JWKS endpoint base
JWT Secret (HS256)	—	—	Not used. Legacy verification only

Golden rule: anything prefixed `NEXT_PUBLIC_*` is embedded into the browser bundle. The **service_role** key must never be a `NEXT_PUBLIC_*` variable, and must never appear in frontend code.

Verify a key's role without exposing it (decodes the unsigned payload):

```
import jwt # PyJWT
payload = jwt.decode(KEY, options={"verify_signature": False})
print(payload["role"]) # -> "anon" or "service_role"
```

3. Supabase dashboard setup

1. **Authentication** → **Providers** → enable **Email**. For local dev, consider turning **off** "Confirm email" so signup logs you straight in. (Leave it on for production and configure the redirect URL in step 4b.)
2. **Project Settings** → **API** → copy the **Project URL**, the **anon** key, and the **service_role** key.
3. **Project Settings** → **JWT Keys** → confirm the **Current signing key** (e.g. `ECC (P-256)`). Nothing to configure here — just know it means **verify via JWKS**. Ignore "Legacy JWT Secret."

4. Frontend

4a. Install dependencies

```
npm install @supabase/supabase-js @supabase/ssr
# If using shadcn/ui for the auth pages:
npm install @radix-ui/react-label class-variance-authority clsx tailwind-merge lucide-react
# shadcn components used: button, input, label, card
```

4b. Environment (`frontend/.env.local`)

```
NEXT_PUBLIC_SUPABASE_URL=https://<project-ref>.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=<anon public key>
NEXT_PUBLIC_API_URL=http://localhost:8000
```

4c. Supabase clients

`lib/supabase/client.ts` (browser):

```
import { createBrowserClient } from "@supabase/ssr"

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
}
```

`lib/supabase/server.ts` (Server Components / Route Handlers):

```
import { createServerClient } from "@supabase/ssr"
import { cookies } from "next/headers"

export async function createClient() {
  const cookieStore = await cookies() // async in Next 15/16
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() { return cookieStore.getAll() },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options))
          } catch { /* called from a Server Component – middleware refreshes cookies */ }
        },
      },
    }
  )
}
```

4d. Session refresh + route guard

`lib/supabase/middleware.ts` :

```

import { createServerClient } from "@supabase/ssr"
import { NextResponse, type NextRequest } from "next/server"

export async function updateSession(request: NextRequest) {
  let supabaseResponse = NextResponse.next({ request })

  const url = process.env.NEXT_PUBLIC_SUPABASE_URL
  const key = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY
  if (!url || !key) return supabaseResponse // not configured yet – don't block

  const supabase = createServerClient(url, key, {
    cookies: {
      getAll() { return request.cookies.getAll() },
      setAll(cookiesToSet) {
        cookiesToSet.forEach(({ name, value }) => request.cookies.set(name, value))
        supabaseResponse = NextResponse.next({ request })
        cookiesToSet.forEach(({ name, value, options }) =>
          supabaseResponse.cookies.set(name, value, options))
      },
    },
  })

  // Do NOT run code between createServerClient and getUser().
  const { data: { user } } = await supabase.auth.getUser()

  if (!user && request.nextUrl.pathname.startsWith("/app")) {
    const to = request.nextUrl.clone()
    to.pathname = "/login"
    to.searchParams.set("redirect", request.nextUrl.pathname)
    return NextResponse.redirect(to)
  }
  if (user && ["/login", "/signup"].includes(request.nextUrl.pathname)) {
    const to = request.nextUrl.clone(); to.pathname = "/app"; to.search = ""
    return NextResponse.redirect(to)
  }
  return supabaseResponse
}

```

Next.js 16 note: the `middleware.ts` file convention is **deprecated in favor of `proxy.ts`**, exporting a function named `proxy` (not `middleware`). Older Next versions still use `middleware.ts` / `middleware`.

`proxy.ts` (repo root, next to `app/`):

```
import { type NextRequest } from "next/server"
import { updateSession } from "@lib/supabase/middleware"

export async function proxy(request: NextRequest) {
  return await updateSession(request)
}

export const config = {
  matcher: ["/(?!_next/static|_next/image|favicon.ico|.*\\.?(?:svg|png|jpg|jpeg|gif|webp|ico)$).*(.*)"],
}
```

4e. Attach the token to backend calls

`lib/supabase/token.ts`:

```
import { createClient } from "../client"

export async function authHeader(): Promise<Record<string, string>> {
  try {
    const supabase = createClient()
    const { data } = await supabase.auth.getSession()
    const token = data.session?.access_token
    return token ? { Authorization: `Bearer ${token}` } : {}
  } catch { return {} }
}
```

Then merge `...(await authHeader())` into the headers of **every** fetch to the backend — regular JSON calls, file uploads, and SSE/streaming requests alike.

4f. Login / signup pages

Client components under `app/login/page.tsx` and `app/signup/page.tsx`. Core calls:

```
const supabase = createClient()

// Login
const { error } = await supabase.auth.signInWithPassword({ email, password })

// Signup (emailRedirectTo only matters if "Confirm email" is ON)
const { data, error } = await supabase.auth.signUp({
  email, password,
  options: { emailRedirectTo: `${window.location.origin}/auth/callback` },
})

// If data.session is null, email confirmation is required – show "check your email".
```

On success: `router.push("/app"); router.refresh()`.

If a page reads `useSearchParams()` (e.g. the `?redirect=` param on login), wrap it in a `<Suspense>` boundary or `next build` will fail static generation.

4g. Auth callback (email confirmation / OAuth)

`app/auth/callback/route.ts` :

```
import { NextResponse } from "next/server"
import { createClient } from "@lib/supabase/server"

export async function GET(request: Request) {
  const { searchParams, origin } = new URL(request.url)
  const code = searchParams.get("code")
  const next = searchParams.get("next") ?? "/app"
  if (code) {
    const supabase = await createClient()
    const { error } = await supabase.auth.exchangeCodeForSession(code)
    if (!error) return NextResponse.redirect(`${origin}${next}`)
  }
  return NextResponse.redirect(`${origin}/login`)
}
```

4h. Sign out

```
const supabase = createClient()
await supabase.auth.signOut()
router.push("/login"); router.refresh()
```

5. Backend (FastAPI)

5a. Dependencies (`requirements.txt`)

```
PyJWT==2.9.0
cryptography>=43.0.0 # required by PyJWT to verify ES256 (asymmetric keys)
```

5b. Config (`app/config.py`)

```
SUPABASE_URL = os.getenv("SUPABASE_URL", "")
SUPABASE_KEY = os.getenv("SUPABASE_KEY", "") # service_role key
# Optional LEGACY fallback only. Verification uses JWKS (SUPABASE_URL); no secret needed.
SUPABASE_JWT_SECRET = os.getenv("SUPABASE_JWT_SECRET", "")
```

5c. Token verification — the key file (`app/auth.py`)

```
import asyncio
import jwt
from jwt import PyJWKClient
from fastapi import Header, HTTPException
from app.config import SUPABASE_URL, SUPABASE_JWT_SECRET

class AuthUser:
    def __init__(self, id: str, email: str | None = None):
        self.id = id
        self.email = email

_jwks_client: PyJWKClient | None = None

def _jwks() -> PyJWKClient:
    global _jwks_client
    if _jwks_client is None: # caches fetched keys after the first call
        _jwks_client = PyJWKClient(f"{SUPABASE_URL}/auth/v1/.well-known/jwks.json")
    return _jwks_client

def _verify(token: str) -> dict:
    alg = jwt.get_unverified_header(token).get("alg", "")
    if alg == "HS256": # legacy fallback only
        if not SUPABASE_JWT_SECRET:
            raise HTTPException(status_code=401, detail="Invalid authentication token.")
        return jwt.decode(token, SUPABASE_JWT_SECRET, algorithms=["HS256"],
audience="authenticated")
    signing_key = _jwks().get_signing_key_from_jwt(token) # asymmetric (ES256/RS256)
    return jwt.decode(token, signing_key.key, algorithms=["ES256", "RS256"],
audience="authenticated")

async def get_current_user(authorization: str | None = Header(default=None)) -> AuthUser:
    if not authorization or not authorization.lower().startswith("bearer "):
        raise HTTPException(status_code=401, detail="Not authenticated. Please sign in.")
    token = authorization.split(" ", 1)[1].strip()
    try:
        payload = await asyncio.to_thread(_verify, token) # keep blocking JWKS off the event loop
    except HTTPException:
        raise
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail="Your session expired. Please sign in again.")
    except jwt.PyJWTError:
        raise HTTPException(status_code=401, detail="Invalid authentication token.")
```

```

except Exception:
    raise HTTPException(status_code=401, detail="Could not verify authentication token.")
user_id = payload.get("sub")
if not user_id:
    raise HTTPException(status_code=401, detail="Authentication token is missing a subject.")
return AuthUser(id=user_id, email=payload.get("email"))

```

Two details that matter:

- **audience="authenticated"** — Supabase user tokens carry **aud: "authenticated"** . Omitting this makes **decode** fail.
- **asyncio.to_thread** — the JWKS fetch and decode are synchronous; run them in a thread so they don't block the async event loop.

5d. Enforce ownership

Make session lookup depend on the authenticated user and compare ids:

```

from fastapi import Depends, Header, HTTPException
from app.auth import get_current_user, AuthUser

async def get_current_session(
    x_session_id: str = Header(...),
    user: AuthUser = Depends(get_current_user),
) -> Session:
    session = await run_db(get_session, x_session_id)
    if not session:
        raise HTTPException(status_code=400, detail="Session not found.")
    if str(session.user_id) != user.id:
        raise HTTPException(status_code=403, detail="You don't have access to this session.")
    ...
    return session

```

Also:

- **On create** (e.g. dataset upload): require **get_current_user** and store **user_id = user.id** .
- **Endpoints that take an id in the path** (state, messages, ...): add **Depends(get_current_user)** and check **str(session.user_id) == user.id** , else **404** .
- **Fix IDOR**: when an endpoint mutates by a path id but also resolves an owned session, operate on the **owned** id, never the raw path param.
- **sendBeacon endpoints** (e.g. a page-unload "close"): **navigator.sendBeacon cannot set headers**, so it can't carry the token. Either leave such an endpoint unauthenticated (only acceptable if it acts on an unguessable id and does something harmless) or redesign it.

6. Database migration (**user_id** + RLS)

Run in **Supabase → SQL Editor**.

Before enabling RLS: confirm the backend connects with the **service_role** key (it bypasses RLS). If your backend uses the *anon* key, RLS will lock it out — switch to *service_role* first.

```
-- Part A (REQUIRED): link rows to users
alter table public.sessions
  add column if not exists user_id uuid references auth.users(id) on delete cascade;
create index if not exists sessions_user_id_idx on public.sessions(user_id);

-- Part B (RECOMMENDED): row-level security (defense in depth)
alter table public.sessions enable row level security;
-- ...repeat: enable RLS on each child table (messages, artifacts, ...)

create policy "own sessions" on public.sessions
  for all using (auth.uid() = user_id) with check (auth.uid() = user_id);

-- child tables are owned transitively through their session:
create policy "own messages" on public.messages
  for all using (exists (
    select 1 from public.sessions s
    where s.id = messages.session_id and s.user_id = auth.uid()
  ));
-- ...repeat the pattern for each child table.
```

Cutover note: existing rows created before the migration have `user_id = NULL` and become inaccessible (`403`) once ownership is enforced. Expected.

7. Acceptance test

1. Visit `/app` while signed out → should redirect to `/login`.
 2. Sign up → land on `/app` (or "check your email" if confirmation is on).
 3. Do a real authenticated action (upload, query) → succeeds.
 4. Sign out → `/app` redirects to `/login` again.
 5. Backend rejects a request with no/expired/garbage token → `401`.
-

8. Troubleshooting

Symptom	Likely cause / fix
Every backend call returns <code>401</code> with a valid login	Still verifying with HS256 + JWT secret . Switch to JWKS/ES256 (§5c).
<code>401</code> "Could not verify" only for some users	JWKS cache stale after a key rotation — restart backend, or the token's <code>kid</code> isn't in JWKS.
Backend DB reads/writes fail after enabling RLS	Backend isn't using the service_role key.
<code>Invalid audience</code>	Missing <code>audience="authenticated"</code> in <code>jwt.decode</code> .
<code>next build</code> fails on login page	<code>useSearchParams()</code> not wrapped in <code><Suspense></code> .
Token never reaches backend	<code>authHeader()</code> not merged into that particular fetch (check upload + SSE).
Signup "works" but no session	"Confirm email" is ON — user must click the emailed link (<code>/auth/callback</code>).

9. File manifest

Frontend

<code>lib/supabase/client.ts</code>	browser client
<code>lib/supabase/server.ts</code>	server client
<code>lib/supabase/middleware.ts</code>	<code>updateSession()</code> – refresh + guard
<code>lib/supabase/token.ts</code>	<code>authHeader()</code> for backend calls
<code>proxy.ts</code>	Next 16 proxy (was <code>middleware.ts</code>)
<code>app/login/page.tsx</code>	sign in
<code>app/signup/page.tsx</code>	sign up
<code>app/auth/callback/route.ts</code>	email-confirm / OAuth exchange
<code>components/auth/AccountMenu</code>	email + sign out
<code>components/ui/{button,input,label,card}.tsx</code>	shadcn primitives
<code>.env.local</code>	<code>NEXT_PUBLIC_SUPABASE_*</code> + API url

Backend

<code>app/auth.py</code>	<code>get_current_user</code> – JWKS verification
<code>app/config.py</code>	<code>SUPABASE_URL</code> / <code>KEY</code> (<code>service_role</code>) / (optional) JWT secret
<code>app/deps.py</code>	<code>get_current_session</code> – ownership check
<code>app/routers/session.py</code>	upload stamps <code>user_id</code> ; ownership on state/messages; IDOR fix
<code>app/db/models.py</code>	<code>Session.user_id</code>
<code>app/db/sessions.py</code>	<code>create_session(user_id=...)</code>
<code>requirements.txt</code>	PyJWT + cryptography
<code>migrations/00X_auth.sql</code>	<code>user_id</code> column + RLS

One-line reminder to carry forward

Verify Supabase user tokens via the JWKS endpoint (asymmetric ES256). The legacy HS256 "JWT Secret" is not used to sign login tokens anymore — do not put it in the backend.